# Week 9: Advanced TypeScript Concepts

FH University of Applied Sciences
TECHNIKUM WIEN

# Week 9: Advanced TypeScript Concepts

## Topics for This Week

1. Advanced TypeScript Features
2. Design Patterns in TypeScript
3. Modular Programming for Scalability
4. Extending Applications with New Features

# 1. Advanced TypeScript Features

## Intersection Types

- Combine multiple types into a single composite type.

```
interface User {
    id: number;
    name: string;
}

interface Admin {
    permissions: string[];
}

type AdminUser = User & Admin;

const admin: AdminUser = {
    id: 1,
    name: "Alice",
    permissions: ["read", "write"]
};
```

# Discriminated Unions

- Use a shared property to narrow down types.

```
type Shape =
    | { kind: "circle"; radius: number }
    | { kind: "rectangle"; width: number; height: number };

function getArea(shape: Shape): number {
    switch (shape.kind) {
        case "circle":
            return Math.PI * shape.radius ** 2;
        case "rectangle":
            return shape.width * shape.height;
    }
}
```

# Generics

- Create reusable and type-safe components.

```
function merge<T, U>(obj1: T, obj2: U): T & U {
    return { ...obj1, ...obj2 };
}

const user = merge({ name: "Alice" }, { age: 30 });
console.log(user); // { name: "Alice", age: 30 }
```

# Structural Typing

- Type compatibility based on structure, not explicit inheritance.

```
interface Point {
    x: number;
    y: number;
}

function logPoint(p: Point): void {
    console.log(`${p.x}, ${p.y}`);
}

const coordinate = { x: 10, y: 20, z: 30 };
logPoint(coordinate); // Works because coordinate has x and y.
```

# 2. Design Patterns in TypeScript

## Singleton Pattern

- Ensure a class has only one instance.

```
class Singleton {
    private static instance: Singleton;

    private constructor() {}

    static getInstance(): Singleton {
        if (!Singleton.instance) {
            Singleton.instance = new Singleton();
        }
        return Singleton.instance;
    }
}

const instance = Singleton.getInstance();
```

# Factory Pattern

- Abstract object creation.

```
class User {
    constructor(public name: string) {}
}

class UserFactory {
    static createUser(name: string): User {
        return new User(name);
    }
}

const user = UserFactory.createUser("Alice");
```

# 3. Modular Programming for Scalability

## Principles of Modular Design

1. **Separation of Concerns**
   - Each module handles a specific functionality.
2. **Reusability**
   - Modules can be used across applications.
3. **Extensibility**
   - New features can be added without modifying existing code.

# Example: Modular Group Chat

## Core Modules:

1. **GroupManager**:
    - Handles group creation and membership.
2. **GroupChatUI**:
    - Renders group chat interfaces.

# 4. Extending Applications with New Features

## Adding Group Chats to the Chat Application

### Step 1: Define Group Management

```
interface Group {
    id: number;
    name: string;
    members: User[];
}

class GroupManager {
    private groups: Group[] = [];
    createGroup(name: string): Group {
        const group = { id: this.groups.length + 1, name, members: [] };
        this.groups.push(group);
        return group;
    }
    addUserToGroup(groupId: number, user: User): void {
        const group = this.groups.find(g => g.id === groupId);
        if (group) {
            group.members.push(user);
        }
    }
}
```

# Step 2: Render Groups in the UI

```
class GroupChatUI {
    constructor(private groupManager: GroupManager) {}

    renderGroups(container: HTMLElement): void {
        container.innerHTML = this.groupManager.getGroups()
            .map(group => `<div>${group.name}</div>`)
            .join('');
    }
}
```

# Future Enhancements

1. **Real-Time Messaging**
   - Use WebSockets for real-time updates.
2. **Authentication**
   - Add user login/logout functionality.
3. **Persistent Data**
   - Save groups and messages using a backend API.

# Weekly Exercise

## Task: Extend the Chat Application with Group Chats

1. Implement `GroupManager` to handle groups and members.
2. Extend the UI to display groups and their messages.
3. Associate messages with specific groups.

# Summary and Q&A

## Key Takeaways

- Advanced TypeScript features (e.g., Intersection Types, Generics).
- Design patterns for scalable applications.
- Modular programming principles.
- Practical application: Group Chat extension.

**Questions?**